



Vibe Coding: It's A Legitimate Strategy

Using AI for Clean, Effective, Up-to-Date, and Secure Code

A presentation by Kali Jackson
(@radicalkjax)

About Kali



@radicalkjax
radicalkjax.com

- Originally from Merced and grew up on a dairy
- Graduated from CSU Stanislaus
- Worked at Geek Squad while going to school
- I've been doing corpo things for about 6 years
- I've been modding/making/hacking since I was 10
- Enjoy lots of outside time with my gf and puppies
- Huge Pokemon, Sailor Moon and Cowboy Bebop fan

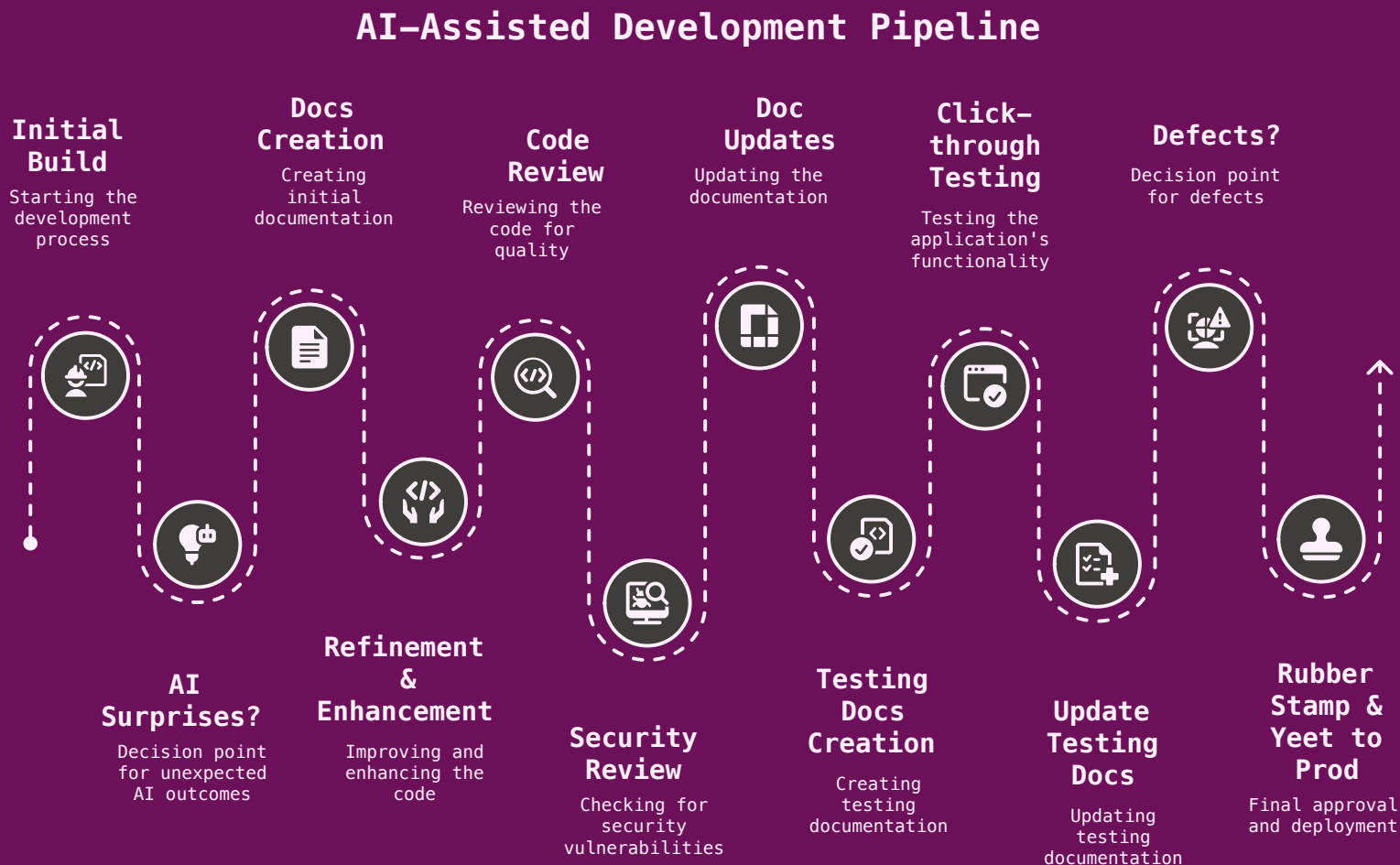


Overview

- The AI-Assisted Development Pipeline
- Different Perspectives on the Pipeline
- Documentation Responsibilities
- Impacts to Users
- Addressing Common Concerns
- Conclusion



The AI-Assisted Development Pipeline





1. Initial Build

Using AI to generate the initial code structure and implementation.

Key Considerations:

- Be specific about requirements
- Provide context about your project
- Specify coding standards and patterns
- Give strong educational and experience background
- Develop strong need to document well
- Give agent artistic style

Effective Prompts:

```
Hi [agent], my name is [your name]. [Build agent worker profile] [build agent educational background: degrees and certs] [minor technical skills][build agent experience: github] [build agent need to document well and visually]
```

```
[Initial task][give agent resources for task]
```

```
[build agent visual style: give examples such as images, websites or repos]
```

```
[build agent art style: give it context such as 'whimsical']
```

```
Stop and let me know once you've completed your thorough research and are ready to begin the project.
```



2. AI Surprises? (Decision Making Time)

Evaluating unexpected AI suggestions and deciding which to incorporate.

Questions to Ask:

- Does this suggestion improve the code?
- Is it aligned with our architecture?
- Does it introduce new dependencies?
- Is it secure and maintainable?

Action Items:

- Document surprising but useful patterns
- Create standards for AI-suggested code



3. Docs Creation

Using AI to help generate initial documentation.

Documentation Types:

- Code comments
- README files
- API documentation
- User guides

Effective Prompts:

Create documentation for this [code/function/API].
Include purpose, parameters, return values, examples,
and potential edge cases.

You're also amazing at creating easily
digestible deep technical documentation for
your projects and favor using mermaid charts
for visual representation.



4. Refinement & Enhancement Cycling

Iteratively improving code with AI assistance.

Best Practices:

- Focus on one improvement at a time
- Verify each change before moving to the next
- Update documentation with each significant change

Example Prompts:

```
-I'm noticing our mermaid diagrams aren't working. Can you see if you can resolve this?  
-Here's a forum where they discuss the issue and resolved it. Take a look to find  
a solution. Don't just view the webpage. Find a more creative way to scrub the  
website.  
  
https://github.com/orgs/marp-team/discussions/207#discussioncomment-1594903  
  
-Lets move on past mermaid diagrams. Lets not use them for this presentation  
  
-Instead lets create a blog post along with our presentation so that we can  
preserve our mermaid diagrams. Look at /blog/_posts/2025-04-21-deep-learning-for-  
malware-analysis.md for an example of how to write it.  
  
-Great, now lets go back to our presentation and remove the mermaid diagrams and  
instead generate place holder content. The placeholder content should be a prompt  
to give napkin.ai so that it can build the diagrams for us.
```



5. Code Review

Using AI to assist in code review process.

Benefits:

- Catch issues human reviewers might miss
- Standardize review process
- Educate developers on best practices

Effective Prompts:

```
-Hi [agent], my name is [your name]. [Principle Engineer] [Masters in Software Engineering] [CSM Cert] [minor technical skills] [build agent experience: github] [build agent need to document well and visually]
```

```
Stop and let me know once you've completed your thorough research and are ready to begin the project.
```

```
-Review this code for potential issues, bugs, or improvements.
```

```
Check if this code follows [specific standards].
```



6. Security Review

Leveraging AI to identify and address security concerns.

Security Checks:

- Input validation
- Authentication/authorization issues
- Data exposure risks
- Dependency vulnerabilities

Effective Prompts:

```
-Hi [agent], my name is [your name]. [Security Engineer] [Masters is Software Engineering] [CSSLP, CompTIA Security+ & CISSP Certs] [minor technical skills] [build agent experience: github] [build agent need to document well and visually]
```

```
Stop and let me know once you've completed your thorough research and are ready to begin the project.
```

```
-Analyze this code for security vulnerabilities.
```

```
How could this code be exploited?
```

```
Suggest security improvements for this implementation.
```



7. Final Doc Updates and Expansion

Ensuring documentation is complete and up-to-date.

Documentation Quality Checklist:

- Accuracy
- Completeness
- Clarity
- Examples
- Troubleshooting guidance

Effective Prompts:

```
-Hi [agent], my name is [your name]. [Technical Writer] [Masters in Software Engineering] [CSM, CISSP, CPCT (Certified Professional Technical Communicator) Certs] [Submitted to pubs like IEEE] [build agent experience: github] [build agent need to document well and visually]
```

```
Stop and let me know once you've completed your thorough research and are ready to begin the project.
```

```
-Update this documentation to reflect the changes made.
```

```
Expand this documentation to include [specific aspects].
```

```
Create a troubleshooting guide for this component.
```



8. Testing Docs Creation

Creating comprehensive test documentation with AI assistance.

Testing Documentation Components:

- Test scenarios
- Expected outcomes
- Edge cases
- Test data requirements
- Testing environment setup

Effective Prompts:

```
-Hi [agent], my name is [your name]. [Technical Writer] [Masters in Software Engineering] [CSM, CISSP, CPCT (Certified Professional Technical Communicator), CSTE (Certified Software Tester), ISTQB Advanced Test Analyst Certs] [Submitted to pubs like IEEE] [build agent experience: github] [build agent need to document well and visually]
```

```
-Create test cases for this [function/component/feature].
```

```
Generate unit tests for this code.
```

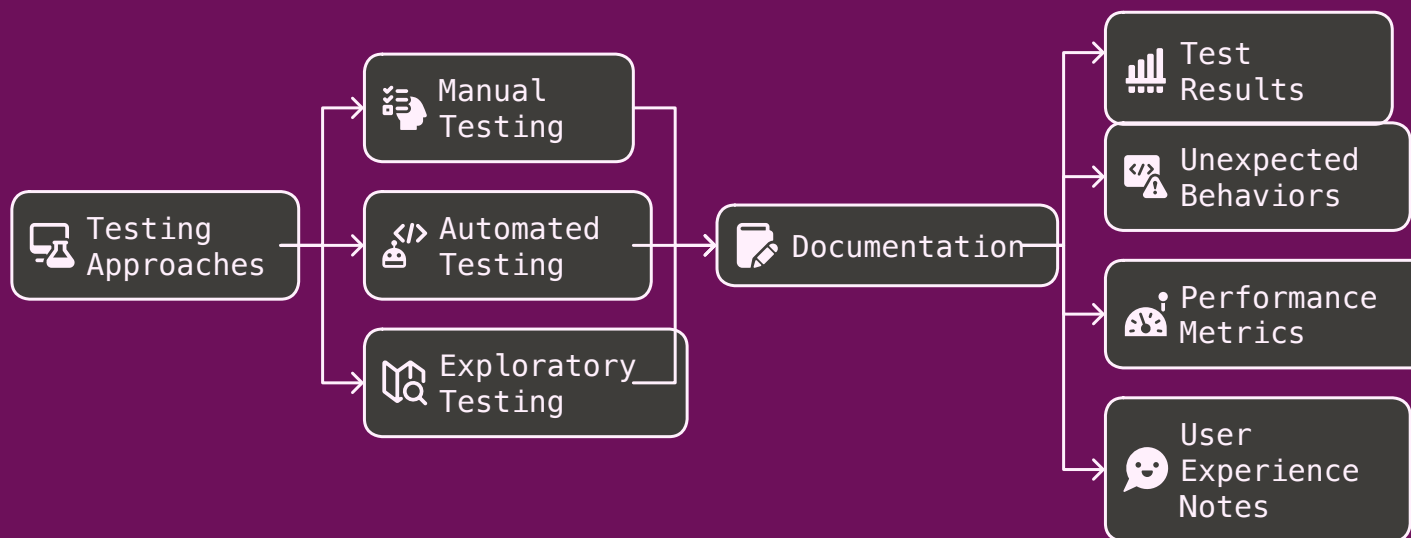
```
What edge cases should be tested for this functionality?
```



9. Click-through Testing

Executing tests and documenting results.

AI-Enhanced Software Testing Process



Made with  Napkin



10. Update Testing Docs

Refining test documentation based on testing results.

Continuous Improvement:

- Add tests for discovered edge cases
- Refine existing test cases
- Document known limitations

Effective Prompts:

Update these test cases based on the issues found.
Generate additional test scenarios for [specific feature].
Create regression tests for the bugs that were fixed.



11. Defects? Back to Refinement

Addressing issues discovered during testing.

Defect Resolution Process:

- Understand the root cause
- Implement fixes
- Verify the fix
- Update documentation
- Add regression tests

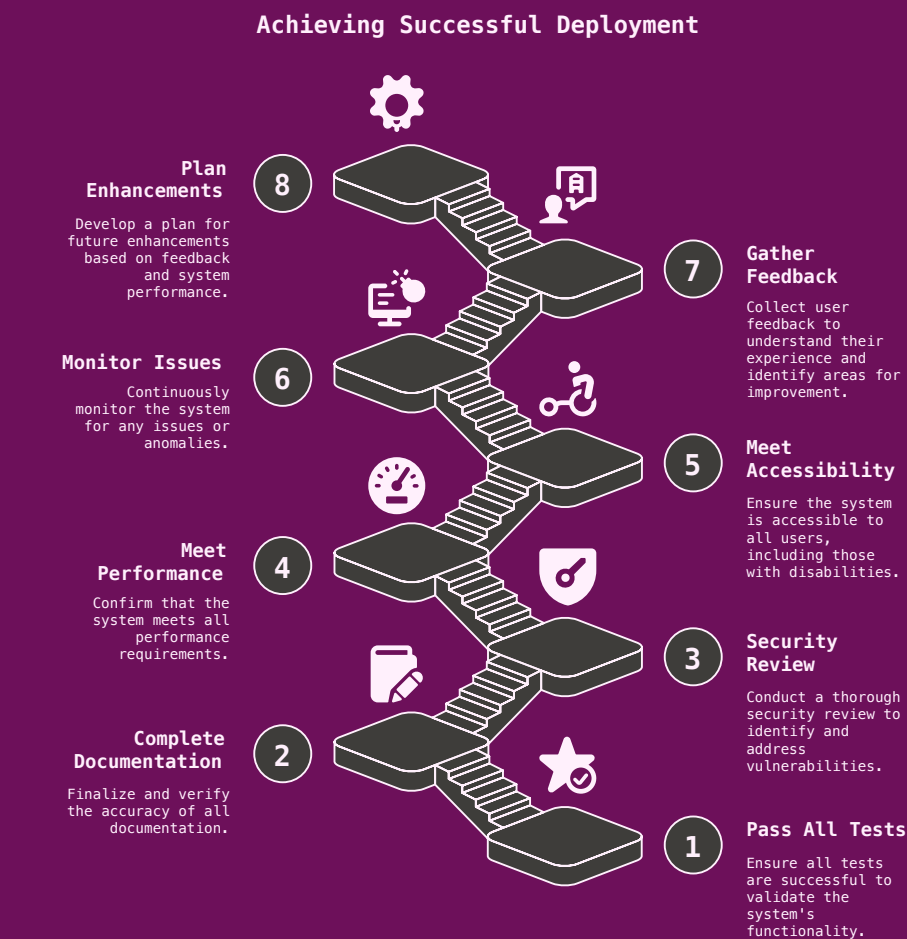
Effective Prompts:

```
Fix this bug: [description of issue].  
Suggest a solution for this problem: [problem details].  
Optimize this code that's causing [specific performance issue].
```



12. Rubber Stamp & Yeet to Prod

Final approval and deployment.





Different Perspectives

Developer

- **Focus:** *Initial Build, AI Surprises, Docs Creation, Refinement, Code Review, Defects, Rubber Stamp & Yeet to Prod*
- **Primary concern:** Functionality and code quality
- **Uses AI for:** Code generation, debugging, optimization

Security Analyst

- **Focus:** *Security Review, Defects, Rubber Stamp & Yeet to Prod*
- **Primary concern:** Identifying vulnerabilities
- **Uses AI for:** Security analysis, threat modeling, compliance checking

Tester

- **Focus:** *Testing Docs Creation, Click-through Testing, Update Testing Docs, Defects*
- **Primary concern:** Finding bugs and edge cases
- **Uses AI for:** Test case generation, regression testing

Designer

- **Focus:** *Initial Build, AI Surprises, Docs Creation, Refinement, Click-through Testing*
- **Primary concern:** User experience and accessibility
- **Uses AI for:** UI patterns, accessibility compliance



Magic Question

Who has the responsibility to create and update documentation?



Everyone

Everyone has a role to play in documentation

Documentation Roles

- **Developers:** Code comments, API docs, architectural decisions
- **Security Analysts:** Security guidelines, vulnerability mitigations
- **Testers:** Test cases, bug reports, user scenarios
- **Designers:** UI guidelines, accessibility requirements, user flows

Benefits of Shared Documentation

- Knowledge transfer between team members
- Building AI knowledge base for future assistance
- Consistent implementation across projects
- Easier onboarding for new team members
- Maintainable codebase with clear context
- Reduced knowledge silos and dependencies



Impacts to the User

Accessibility

- AI can help ensure accessibility standards are met
- Generates accessible code patterns
- Identifies accessibility issues

Thinking Bigger

- Enables tackling more ambitious projects
- Helps understand complex patterns
- Facilitates learning new technologies

Freeing Spoons

- Reduces cognitive load on repetitive tasks
- Allows focus on creative problem-solving
- Handles boilerplate code generation

Effective Communication

- Teaches how to articulate requirements clearly
- Improves technical communication skills
- Encourages precise specification



Impact to User's Wallet

API Costs

- Consider cost per token for AI services
- Strategies to optimize costs:
 - Refine prompts to be concise
 - Batch similar requests
 - Cache common responses
 - Use smaller models for simpler tasks

Cost vs. Value

- Time saved × Developer hourly rate
- Reduced bugs and technical debt
- Faster time to market
- Learning and skill development



Addressing Common Concerns

"But Kali, it's just copying someone else's code! How can I trust it?"

I know most of you are going to Stack Overflow and GitHub to c&p code. Do you trust that code inherently? We've been teaching that's a no-no for a while. You still have a responsibility to maintain your code. AI as a tool doesn't alleviate you from that responsibility.

Key Points:

- AI-generated code requires the same scrutiny as any other code
- You remain responsible for understanding and maintaining your codebase
- AI is a tool, not a replacement for developer judgment
- Verification and testing are still essential



Best Practices for AI-Assisted Coding

Understand Implementation

- Don't blindly copy AI suggestions
- Ask AI to explain complex code
- Review code for security implications

Verify and Test

- Test all AI-generated code
- Verify edge cases
- Implement automated testing

Maintain Standards

- Ensure AI output follows your standards
- Refactor as needed
- Keep code consistent with existing patterns

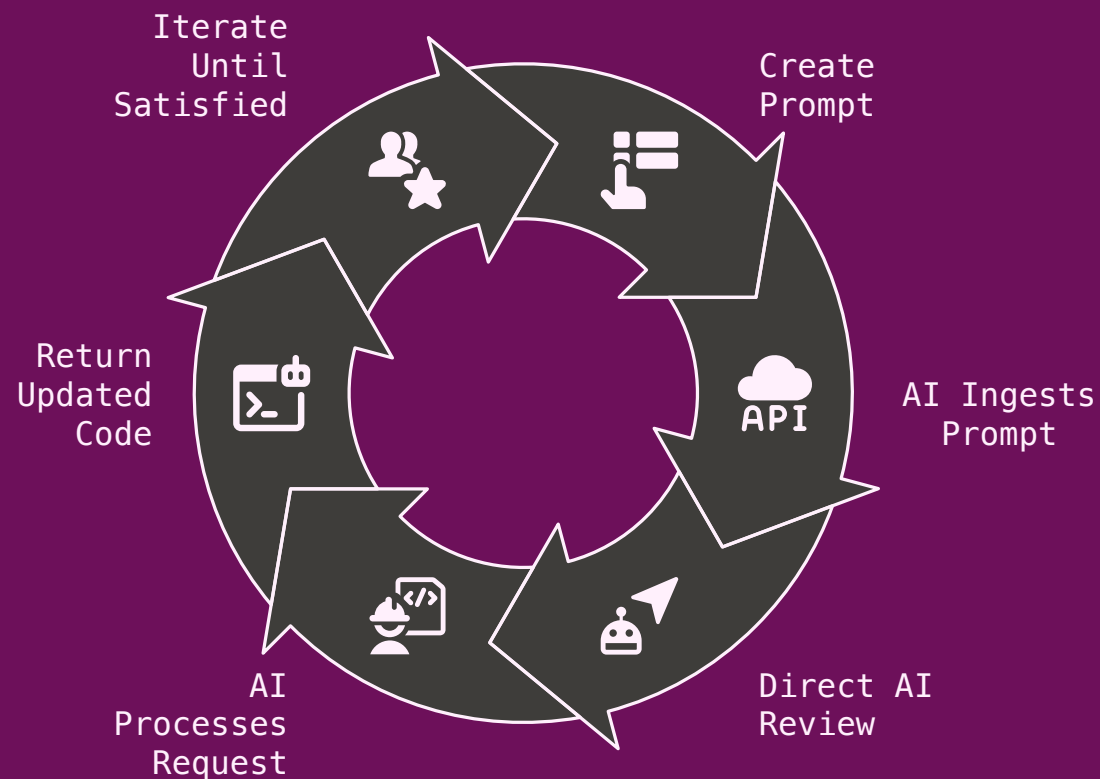
Continuous Learning

- Use AI as a learning tool
- Understand the patterns it suggests
- Share insights with your team



AI Integration in Security

AI-Driven Secure Code Review Cycle



Made with Napkin



Conclusion

AI-assisted Development

- Streamlines the development process
- Enhances productivity across roles
- Requires responsible use and verification
- Doesn't replace human judgment and expertise

Key Takeaways

- The quality of AI output depends on your input
- Documentation remains everyone's responsibility
- AI is a tool that amplifies your capabilities
- Vibe coding is legitimate when done responsibly



Thank You!

Questions?